

MEDIATEK

MT8676 Hypervisor System User Manual

Version: 1.2
Release date: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-11-04	彭凯	正式版
1.1	2024-11-19	彭凯	- 增加全虚拟化 L+A+L 相关描述。 - 增加章节 1.2.3 Thyp sdk 预编译文件、章节 0 - 添加共享文件夹、章节 1.5.1 切换 USB。
1.2	2025-07-11	彭凯	- 完善章节 1.1 名词解释、章节 1.4.1 调整虚拟机内存。 - 增加章节 1.6 开机流程、章节 1.7 关机流程。

目录

版本记录 2

目录 3

图片目录 3

表格目录 4

1 Hypervisor System 5

1.1 名词解释 5

1.2 系统架构 5

1.2.1 project 说明 7

1.2.2 代码结构 7

1.2.3 Thyp sdk 预编译文件 8

1.3 编译与烧录 9

1.3.1 Build Server 9

1.3.1 编译指令 10

1.3.2 软件构成 12

1.3.3 烧录软件 14

1.4 客制化 16

1.4.1 调整虚拟机内存 16

1.4.2 调整虚拟机 CPU 16

1.4.3 添加共享文件夹 17

1.5 调试技巧 18

1.5.1 切换 USB 18

1.5.2 ADB 19

1.5.3 查询虚拟机状态 19

1.6 开机流程 19

1.7 关机流程 21

1.7.1 UOS 关机 21

1.7.2 SOS 关机 22

1.7.3 reboot soc 24

附件一 附加条款 25

图片目录

图 1-1.全虚拟化 L+A+L 系统架构.....6

图 1-2. 部分虚拟化 L+A+L 系统架构.....6

图 1-3 Thyp sdk 预编译文件目录结构.....8

图 1-4. 下载 SP_Flash_Tool.....14

图 1-5. 加载 flash.xml15

图 1-6. 软件烧录.....15

图 1-7. L+A+L 内存布局.....16

图 1-8. 开机流程.....20

图 1-9. UOS 关机流程22

图 1-10. SOS 关机流程.....23

图 1-11. reboot soc 流程24

表格目录

表 1-1. 名词解释.....5

表 1-2. Yocto Hypervisor Project.....7

表 1-3. Hypervisor 代码.....7

表 1-4 Thyp sdk 预编译文件说明.....8

表 1-5. Yocto build server 要求.....10

表 1-6. Android build server 要求.....10

表 1-7. SOS 镜像文件.....12

表 1-8. UOS Android 镜像文件.....13

表 1-9. UOS T-Box 镜像文件.....13

表 1-10. adb shell 指令.....19

1 Hypervisor System

MT8676 Hypervisor SDK 由 Yocto、Android 和 Thyp SDK 三大部分组成。本章节主要介绍 Yocto 和 Android 与 Hypervisor 相关的系统内容。Yocto 单系统的介绍请参考文档 *MT8676_Yocto_System_User_Manual*，Android 单系统的介绍请参考文档 *MT8676_Android_System_User_Manual*。本章节包括系统架构、编译与烧录、客制化、调试工具与技巧等内容。

1.1 名词解释

表 1-1. 名词解释

缩略词	全称及释义
SOS	Service OS。包含了虚拟机管理程序和虚拟化后端的操作系统。
UOS	User OS。包含了虚拟化前端的操作系统。
Nebula	谦川 Hypervisor 的微内核。
全虚拟化	除 WiFi、BT、GPS、Tbox、GPIO、USB 外，其余模块全部虚拟化。
部分虚拟化	与全虚拟化相比，APU、GPU、Camera、Codec 等模块不虚拟化，直通 Android。

1.2 系统架构

MT8676 Hypervisor 支持两种架构，一种是全虚拟化 L+A+L 架构，如图 1-1 所示。一种是部分虚拟化 L+A+L 架构，如图 1-2 所示。这两种架构都由四大块组成。第一个是谦川的 Hypervisor 和 Nebula 微内核。第二个是 SOS，作为主机，主要运行虚拟机管理程序和虚拟化后端驱动。第三个是 UOS Android，作为客户机，主要运行虚拟化前端驱动和部分直通硬件的驱动。第四个是 UOS T-Box，作为客户机，主要运行 T-Box 和 GPS 的驱动。其中 SOS 和 UOS T-Box 是 Linux 系统，通过 Yocto 构建框架编译。UOS Android 是 Android 系统。

对比全虚拟化架构和部分虚拟化架构，主要区别在于 APU、GPU、Camera、Codec 等模块，由虚拟化变为 UOS Android 直通。

MT8676 只有一个 USB 硬件，所以某一时刻只能直通到某一个操作系统。对于全虚拟化架构，默认直通到 SOS。对于部分虚拟化架构，默认直通到 UOS Android。切换 USB 的方法请参考 1.5.1 小节。

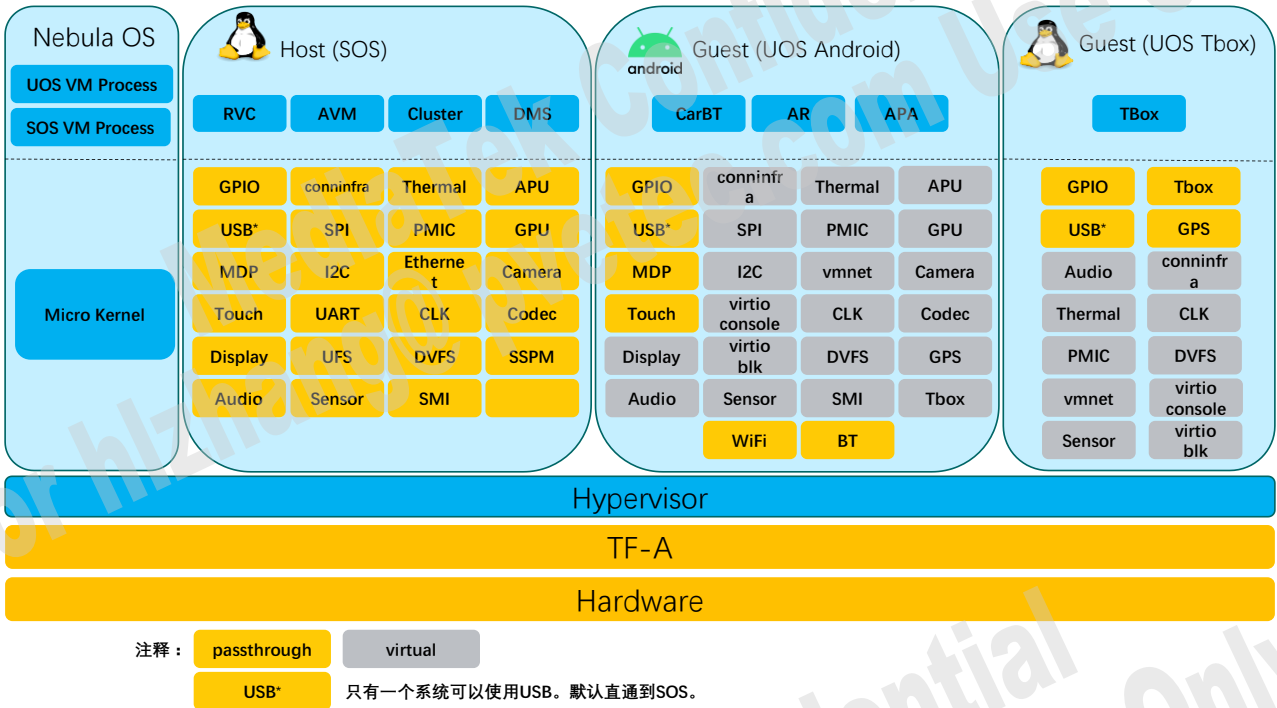


图 1-1.全虚拟化 L+A+L 系统架构

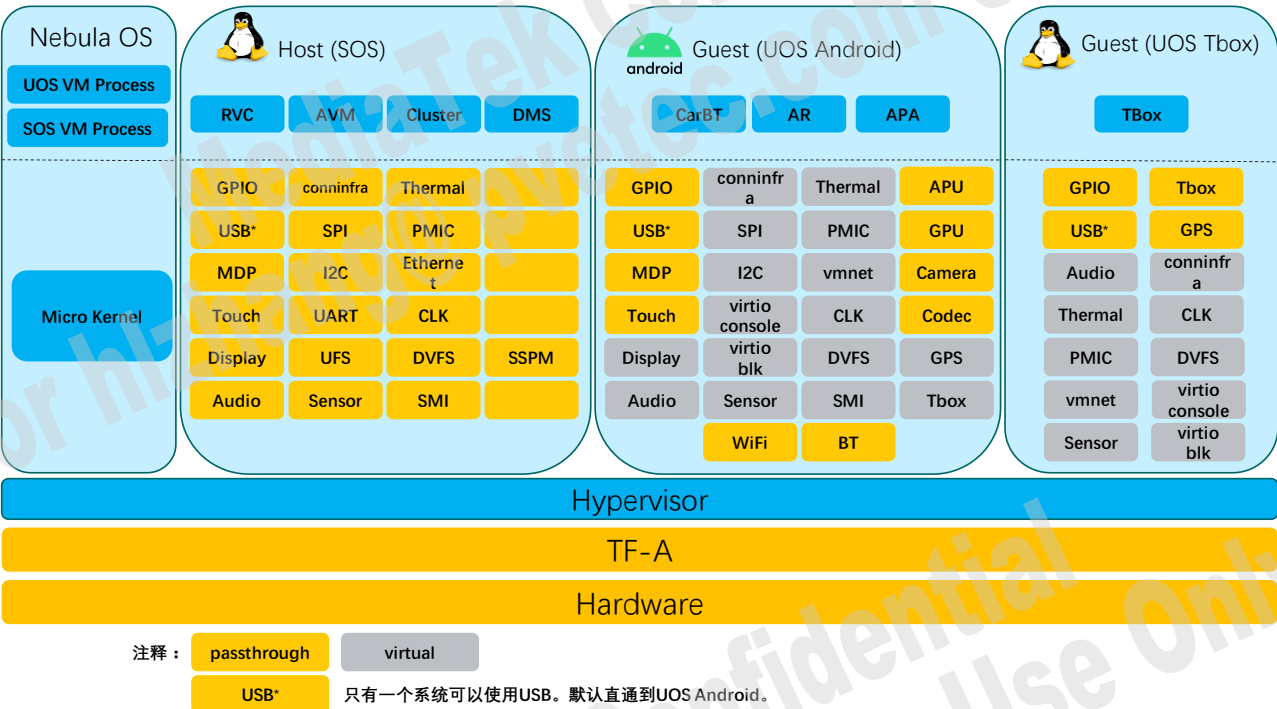


图 1-2. 部分虚拟化 L+A+L 系统架构

1.2.1 project 说明

表 1-2 列出了 MT8676 Hypervisor SDK 提供的 Yocto project 和 Android project，总共两组。请按表格所列搭配使用，不可混搭。

在下文中，SOS 指 auto8676p1_64_hyp 或 auto8676p1_64_hyp_sos。UOS Android 指 auto8676p1_64_bsp_vm 或 auto8676p1_64_bsp_vm_tbox。UOS T-Box 指 auto8676p1_64_uos_tbox。

表 1-2. Yocto Hypervisor Project

Yocto (SOS)	Yocto (UOS T-Box)	Android (UOS Android)	说明
auto8676p1_64_hyp	auto8676p1_64_uos_tbox	auto8676p1_64_bsp_vm	全虚拟化。
auto8676p1_64_hyp_sos	auto8676p1_64_uos_tbox	auto8676p1_64_bsp_vm_tbox	部分虚拟化。

1.2.2 代码结构

MT8676 Hypervisor 代码结构与 Yocto 单系统和 Android 单系统基本一致，只是在单系统基础上增加了一些 Hypervisor 相关的代码。单系统的代码结构请参考文档 *MT8676_Yocto_System_User_Manual* 和 *MT8676_Android_System_User_Manual*。全虚拟化架构和部分虚拟化架构共用同一份代码，SOS 和 UOS T-Box 共用同一份 Yocto 代码。

表 1-3. Hypervisor 代码

代码库	路径	说明
Yocto	meta/meta-mediatek-mt8676-hyp	MT8676 Yocto 虚拟化层，包含了与虚拟化相关的一些配置文件和 bb。
Yocto	src/kernel/modules/mt8676/virt/grt	Yocto 端的虚拟化驱动。
Yocto	prebuilt/hypervisor/grt_mt8676	存放谦川的 Thyp sdk 预编译文件。
Android	vendor/mediatek/kernel_modules/virt	Android 端的虚拟化驱动。
Android	device/mediateksample/auto8676p1_64_bsp_vm	Android auto8676p1_64_bsp_vm 项目的配置文件。
Android	device/mediateksample/auto8676p1_64_bsp_vm_tbox	Android auto8676p1_64_bsp_vm_tbox 项目的配置文件。

1.2.3 Thyp sdk 预编译文件

Thyp sdk 预编译文件存放在 Yocto 代码库的 `prebuilt/hypervisor/grt_mt8676` 目录，主要包含谦川虚拟机镜像、虚拟机管理程序和虚拟化配置文件等。目录结构如图 1-3 所示。`auto8676p1_64_hyp` 目录存放了全虚拟化架构相关的文件，`auto8676p1_64_hyp_sos` 目录存放了部分虚拟化架构相关的文件，`auto8676p1_64_uos_tbox` 目录存放了 UOS T-Box 的配置文件。关于每个文件的作用及其说明，请参考表 1-4。除 `gz.img` 外，其余文件都是安装到 SOS 的根文件系统。

```
├── auto8676p1_64_hyp
│   ├── gz.img
│   ├── nbl_vmm
│   ├── uos_alps_bootloader_lk2.pb.txt
│   ├── uos_alps_pv8676.lua
│   └── vm_srv_cfg.pb.txt
├── auto8676p1_64_hyp_sos
│   └── ...
├── auto8676p1_64_uos_tbox
│   ├── uos_tbox_bootloader_lk2.pb.txt
│   └── uos_tbox_pv8676.lua
├── check_gl.sh
├── gpu_server
├── libluajit.so
├── libvmm.so
├── nblruncmd
├── nbl_vm_ctl
├── nbl_vm_post_android.sh
├── nbl_vm_post_tbox.sh
├── nbl_vm_pre.sh
├── nbl_vm_srv
├── nbl_vm_srv_post.service
├── nbl_vm_srv.service
├── nc-vsock
├── NebulaProprietary
├── query_input_event.sh
├── symbols
│   └── ...
├── video_server
├── virtiofsd
├── vm_audio_cfg.pb.txt
└── vm_audio_shared_irq.pb.txt
```

图 1-3 Thyp sdk 预编译文件目录结构

表 1-4 Thyp sdk 预编译文件说明

文件或目录	说明	路径
gz.img	谦川 nebula 虚拟机固件。SOS 的配置文件打包在 gz.img 中。	gz_a 分区

文件或目录	说明	路径
nbl_vmm	虚拟机管理程序。对 SOS 而言，一个 UOS 就是一个 nbl_vmm 进程。	/usr/bin
vm_srv_cfg.pb.txt	虚拟机服务配置文件。	/vendor/etc/hyper_android
nbl_vm_srv	虚拟机服务程序。读取 vm_srv_cfg_8676.pb.txt 文件的配置，并根据配置启动 nbl_vmm 进程。	/usr/bin
nbl_vm_srv.service	systemd 服务，用于启动 nbl_vm_srv 进程。	/usr/lib/systemd/system
nbl_vm_pre.sh	用于在虚拟机启动之前执行一些任务。	/vendor/etc/hyper_android
nbl_vm_srv_post.service	systemd 服务，用于在虚拟机启动完成后执行一些任务。	/usr/lib/systemd/system
nbl_vm_post_android.sh	用于 UOS Android 启动后执行一些任务。	/vendor/etc/hyper_android
nbl_vm_post_tbox.sh	用于 UOS T-Box 启动后执行一些任务。	/vendor/etc/hyper_tbox
nbl_vm_ctl	虚拟机控制程序。可以启动、停止虚拟机，也可以查询一些虚拟机的基本信息。	/usr/bin
uos_alps_bootloader_lk2.pb.txt	UOS Android 的 bootloader 文件。用于配置 bootloader 相关信息。	/vendor/etc/hyper_android
uos_alps_pv8676.lua	UOS Android 虚拟机的配置文件。用于配置虚拟机的 CPU、内存、中断等。	/vendor/etc/hyper_android
uos_tbox_bootloader_lk2.pb.txt	UOS T-Box 的 bootloader 文件。	/vendor/etc/hyper_tbox
uos_tbox_pv8676.lua	UOS T-Box 虚拟机的配置文件。	/vendor/etc/hyper_tbox
gpu_server	GPU 虚拟化 host 端处理程序。	/usr/bin
video_server	video 虚拟化 host 端处理程序。	/usr/bin
virtiofsd	virtio 文件系统后端程序。	/usr/bin
symbols	存放 nbl_vmm、nbl_vm_srv、nbl_vm_ctl 等文件的符号文件，用于调试。	N/A

1.3 编译与烧录

1.3.1 Build Server

表 1-5 列出了编译 MT8676 Yocto5.0 对编译主机的要求。表 1-6 列出了编译 MT8676 Android 对编译主机的要求。

表 1-5. Yocto build server 要求

项目	要求
磁盘空间	不小于 300GB
内存	不小于 32GB
发行版本	Ubuntu 20.04, 22.04 等，具体参考 Supported Linux Distributions ^[1] 。
编译主机需要安装的工具	参考 Required Packages for the Build Host ^[2] 。
Git	1.8.3.1 或更高版本
tar	1.28 或更高版本
Python	3.8.0 或更高版本
GNU Make	4.0 或更高版本

表 1-6. Android build server 要求

项目	要求
磁盘空间	不小于 250GB
内存	不小于 24GB
发行版本	Ubuntu 18.04 (推荐)
编译主机需要安装的工具	参考 http://source.android.com/source/initializing.html
Git	1.9.1 或更高版本
Shell	4.4.19 或者更高版本 (Ubuntu18.04 自带)(推荐)
Perl	5.26.1 (Ubuntu18.04 自带)(推荐)
Python	Python 2.7.17 & Python 3.6.9 (Ubuntu18.04 自带)(推荐)
GNU Make	4.1 (Ubuntu18.04 自带)(推荐)

1.3.1 编译指令

对于全虚拟化架构，需要编译 Yocto (SOS) auto8676p1_64_hyp、Yocto (UOS T-Box) auto8676p1_64_uos_tbox 和 Android (UOS Android) auto8676p1_64_bsp_vm。编译指令如下。由于编译过程中会设置一些环境变量，为了避免

[1] <https://docs.yoctoproject.org/ref-manual/system-requirements.html#supported-linux-distributions>

[2] <https://docs.yoctoproject.org/ref-manual/system-requirements.html#required-packages-for-the-build-host>

互相干扰，建议使用三个不同的终端分别编译两个 Yocto 和 Android。为了在同一份 codebase 编译两个不同的 yocto project，给 oe-init-build-env 脚本传递了一个参数，用于指定编译目录的名字。另外，auto8676p1_64_hyp_uos_tbox 的编译目标是 mtk-core-image-auto8676-uos。

SOS 的编译产物在 build-sos/tmp/deploy/images/auto8676p1_64_hyp 目录下。UOS T-Box 的编译产物在 build-uos/tmp/deploy/images/auto8676p1_64_uos_tbox 目录下。Android 的编译产物在 out/target/product/auto8676p1_64_bsp_vm/merged 目录下。

编译完成后，请将表 1-8 和表 1-9 列出的镜像文件，拷贝到 SOS 的编译目录。

```
# 终端 1, Yocto (SOS) auto8676p1_64_hyp
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676
```

```
# 终端 2, Yocto (UOS Tbox) auto8676p1_64_uos_tbox
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_uos_tbox
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos
```

```
# 终端 3, Android auto8676p1_64_bsp_vm
cd path/to/android-codebase
python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py --run
full_auto8676p1_64_bsp_vm-userdebug
```

对于部分虚拟化架构，需要编译 Yocto (SOS) auto8676p1_64_hyp_sos、Yocto (UOS T-Box) auto8676p1_64_uos_tbox 和 Android (UOS Android) auto8676p1_64_bsp_vm_tbox。编译指令如下。

SOS 的编译产物在 build-sos/tmp/deploy/images/auto8676p1_64_hyp_sos 目录下。UOS T-Box 的编译产物在 build-uos/tmp/deploy/images/auto8676p1_64_uos_tbox 目录下。Android 的编译产物在 out/target/product/auto8676p1_64_bsp_vm_tbox/merged 目录下。

编译完成后，请将表 1-8 和表 1-9 列出的镜像文件，拷贝到 SOS 的编译目录。

```
# 终端 1, Yocto (SOS) auto8676p1_64_hyp_sos
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp_sos
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676
```

```
# 终端 2，Yocto (UOS Tbox) auto8676p1_64_uos_tbox
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_uos_tbox
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos
```

```
# 终端 3，Android auto8676p1_64_bsp_vm_tbox
cd path/to/android-codebase
python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py --run
full_auto8676p1_64_bsp_vm_tbox-userdebug
```

1.3.2 软件构成

表 1-7 列出 SOS 的镜像文件，表 1-8 列出了 UOS Android 的镜像文件，表 1-9 列出了 UOS T-Box 的镜像文件。

表 1-7. SOS 镜像文件

文件名	分区	说明
MT6897_Android_scatter.xml	N/A	分区表，由 Yocto ptgen-v2.bb 编译出来。
download_agent/DA_BR.bin	N/A	download agent，用于烧录。由 collect.bb 编译出来。
bl2.img	preloader_a preloader_b	Yocto bootloader，由 Yocto lk2.bb 编译出来。
modem.img	modem_a	modem 固件，由 Yocto modem.bb 编译出来。
spmfw.img	spmfw_a	System Power Management 固件，由 collect.bb 编译出来。
mcf_ota.img	mcf_ota_a	由 Yocto collect.bb 编译出来。
pi_img.img	pi_img_a	由 Yocto collect.bb 编译出来。
dpm.img	dpm_a	由 Yocto collect.bb 编译出来。
scp.img	scp_a	System Companion Processor 固件，由 Yocto tinysys-scp.bb 编译出来。
ccu.img	ccu_a	Camera Control Unit 固件，由 collect.bb 编译出来。
vcp.img	vcp_a	vcp firmware，由 tinysys-vcp.bb 编译出来。
sspm.img	sspm_a	Secure System Power Manager 固件，由 collect.bb 编译出来。
mcupm.img	mcupm_a	power/performance manager 固件，由 collect.bb 编译出来。
gpueb.img	gpueb_a	GPU firmware，由 tinysys-gpueb.bb 编译出来。
apusys.img	apusys_a	APU firmware，由 tinysys-apusys.bb 编译出来。

文件名	分区	说明
gz.img	gz_a	谦川 nebula 虚拟机固件，由 nbl-vmm-hyp.bb 编译出来。
connsys_bt.img	connsys_bt_a	BT 固件，由 collect.bb 编译出来。
connsys_wifi.img	connsys_wifi_a	WIFI 固件，由 collect.bb 编译出来。
connsys_gnss.img	connsys_gnss_a	GNSS 固件，由 gps-fw_1.0.0.bb 编译出来。
logo.img	logo_a	logo，由 makelogo.bb 编译出来。
audio_dsp.img	audio_dsp_a	audio DSP 固件，由 collect.bb 编译出来。
system.ext4	system	Yocto 根文件系统，由 mtk-core-image-auto8676.bb 编译出来。
userdata.ext4	yocto-userdata	Yocto /data 分区，由 mkusrdata.bbclass 编译出来。
bl2-an.img	bl2-an	Android bootloader，由 lk2-an.bb 编译出来。
yocto-boot.img	yocto-boot_a	Yocto Linux kernel，fit 格式，包含 kernel img、dtb 和 initramfs。由 linux-mtk-extension_6.1.bb 编译出来。其中 initramfs 由 core-image-minimal-initramfs.bb 编译出来。

表 1-8. UOS Android 镜像文件

文件名	分区	说明
vbmeta.img	vbmeta_a	system 和 vendor layer 一起的 meta data，由 Android 编译出来。
vbmeta_system.img	vbmeta_system_a	system layer 的 meta data，由 Android 编译出来。
vbmeta_vendor.img	vbmeta_vendor_a	vendor system layer 的 meta data，由 Android 编译出来。
boot.img	boot_a	Android kernel 镜像，由 Android 编译出来。
vendor_boot.img	vendor_boot_a	Android ramdisk ko 镜像，由 Android 编译出来。
init_boot.img	init_boot_a	Android init 镜像，由 Android 编译出来。
dtbo.img	dtbo_a	Android dtb overlay 镜像，由 Android 编译出来。
tee.img	tee_a	ATF，由 Android 编译出来。
super.img	super	Android super 镜像，由 Android 编译出来。
userdata.img	userdata	Android userdata 分区，由 Android 编译出来。

表 1-9. UOS T-Box 镜像文件

文件名	分区	说明
bl2-tbox.img	bl2-tbox_a	UOS T-Box bootloader，由 Yocto lk2.bb 编译出来。

文件名	分区	说明
boot_uos_tbox.img	boot_uos_tbox_a	UOS T-Box Linux kernel, fit 格式, 包含 kernel img 和 dtb。由 linux-mtk-extension-uos_6.1.bb 编译出来。
system_uos_tbox.ext4	system_uos_tbox	UOS T-Box 根文件系统, 由 Yocto mtk-core-image-auto8676-uos.bb 编译出来。
userdata_uos_tbox.ext4	userdata_uos_tbox	UOS T-Box/data 分区, 由 Yocto mkusrdata.bbclass 编译出来。

1.3.3 烧录软件

烧录软件需要用到 Type-C 线和 SP_Flash_Tool 工具。如果没有 SP_Flash_Tool 工具, 请访问 [Online](#)^[3] 网站下载。打开网页后, 搜索 SP_Flash_Tool, 下载最新版本, 如图 1-4 所示。下载完成后解压下载的压缩包。

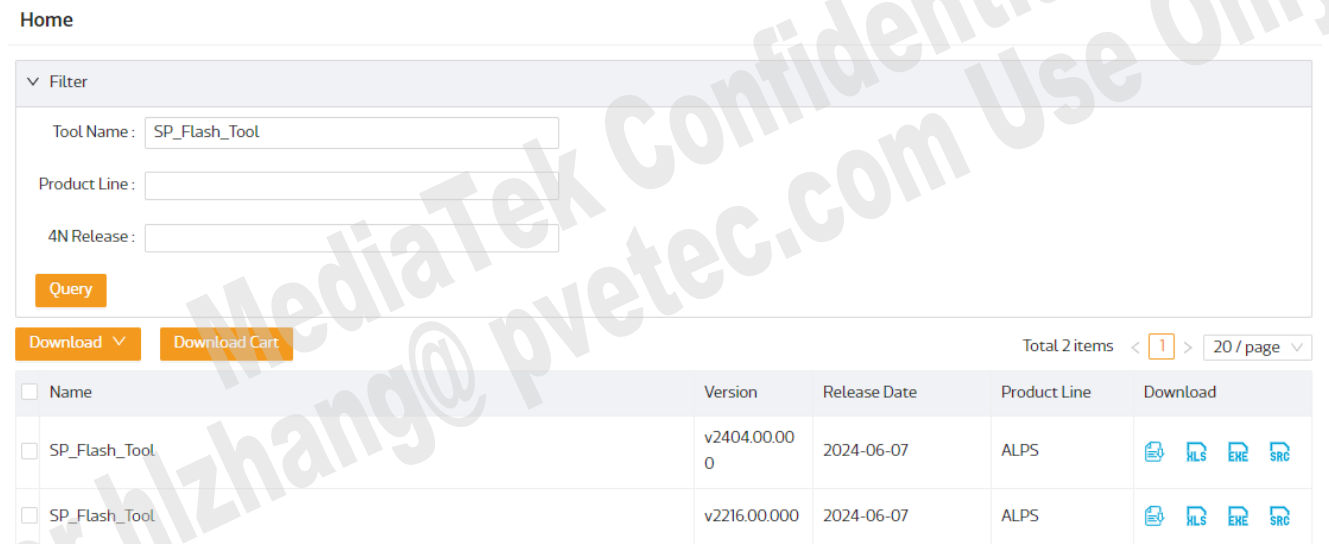


图 1-4. 下载 SP_Flash_Tool

首先用 Type-C 线将平台与电脑连接起来。然后双击 SP_Flash_Tool_V6/SPFlashToolV6.exe 打开烧录工具。第一步选择 Download-XML 文件, 点击右上角的 choose 按钮, 选择软件包下面的 download_agent/flash.xml 文件。第二步在 Download 按钮下方的下拉框中选择 Format All + Download。效果如图 1-5 所示。第三步, 确保平台处于断电状态。第四步, 点击 Download 按钮。第五步, 按住 download key (KPCOLO, SW907) 不松手, 平台上电, 这时会自动开始烧录, 如图 1-6 所示。此时可以松开按键。如果有遇到错误弹窗, 请先将平台断电, 点击 Stop 按钮退出下载模式。确保平台掉电完毕后, 再尝试烧录。

[3] <https://online.mediatek.com/apps/tool/>

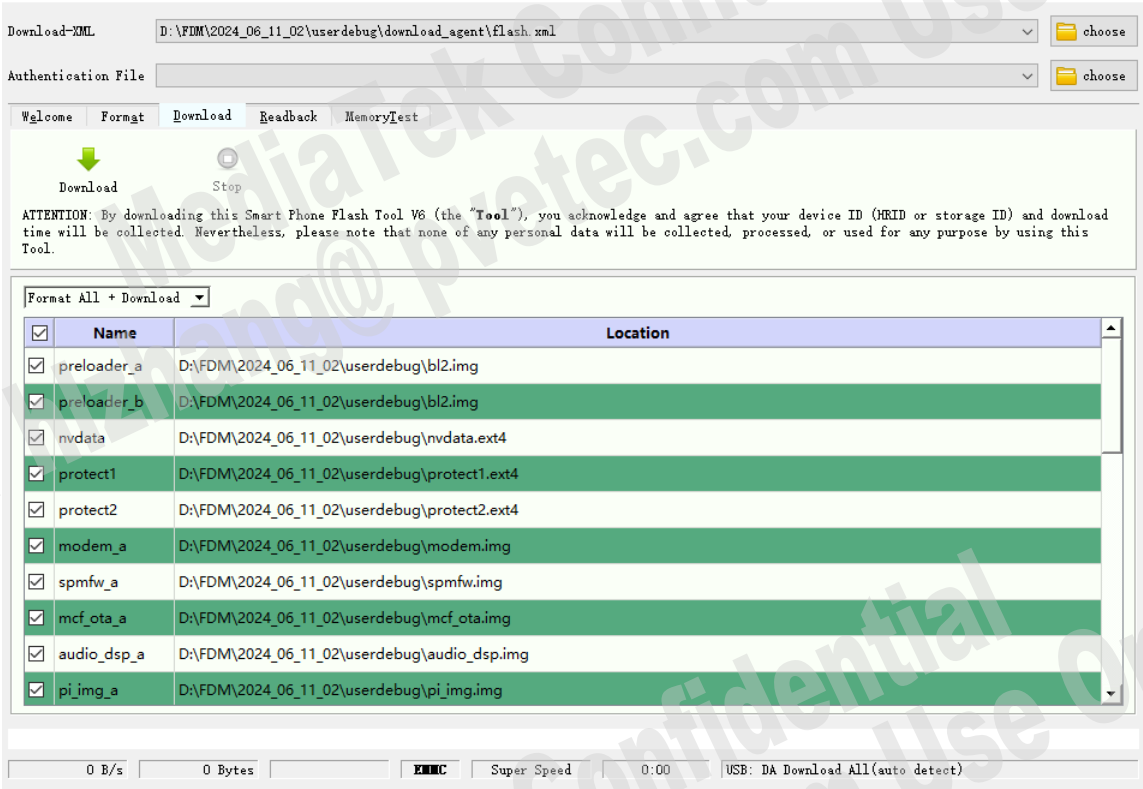


图 1-5. 加载 flash.xml

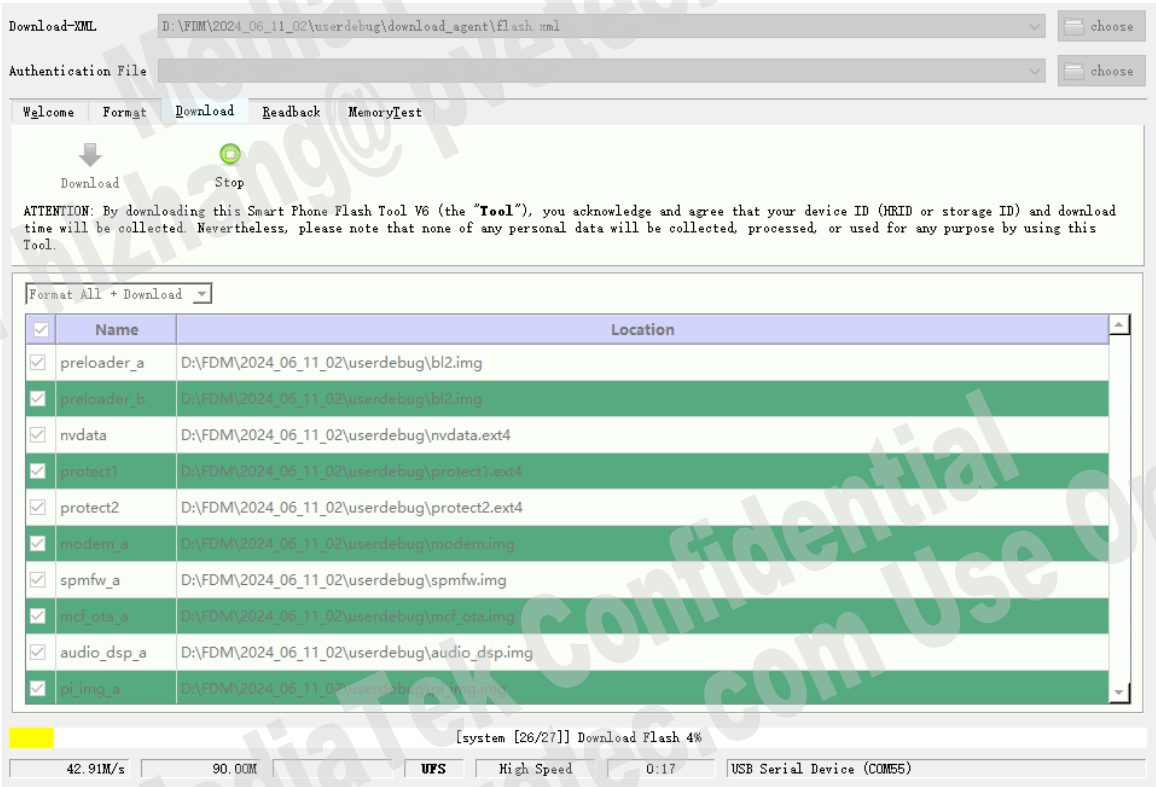


图 1-6. 软件烧录

1.4 客制化

1.4.1 调整虚拟机内存

图 1-7 展示了 L+A+L 架构的内存布局。Yocto 代码库的配置文件 `meta/meta-mediatek-mt8676-hyp/conf/common/memory-size.inc` 决定了每个操作系统的内存大小。变量 `SOS_DRAM_SIZE` 和 `UOS_TBOX_DRAM_SIZE` 分别决定了 SOS 和 UOS Tbox 的内存大小。UOS Android 使用剩余的所有内存，不受某个变量的控制。在公版平台，SOS 默认 6GB，UOS TBox 默认 1GB，UOS Android 默认 9GB。为了保证 SOS 正常运行，全虚拟化方案 `SOS_DRAM_SIZE` 不建议小于 6GB，部分虚拟化方案不建议小于 4GB。

另外，modem、GPS 最大寻址 32bit，Wi-Fi 最大寻址 34bit。目前 modem 和 GPS 是在 SOS 申请内存，可以满足 32bit 的要求。Wi-Fi 是在 UOS Android 申请内存，为了确保 Wi-Fi 能申请到 32bit 的内存，SOS 和 UOS Tbox 的总内存必须小于等于 14GB。关于寻址限制的更多说明，请参考对应模块的文档。

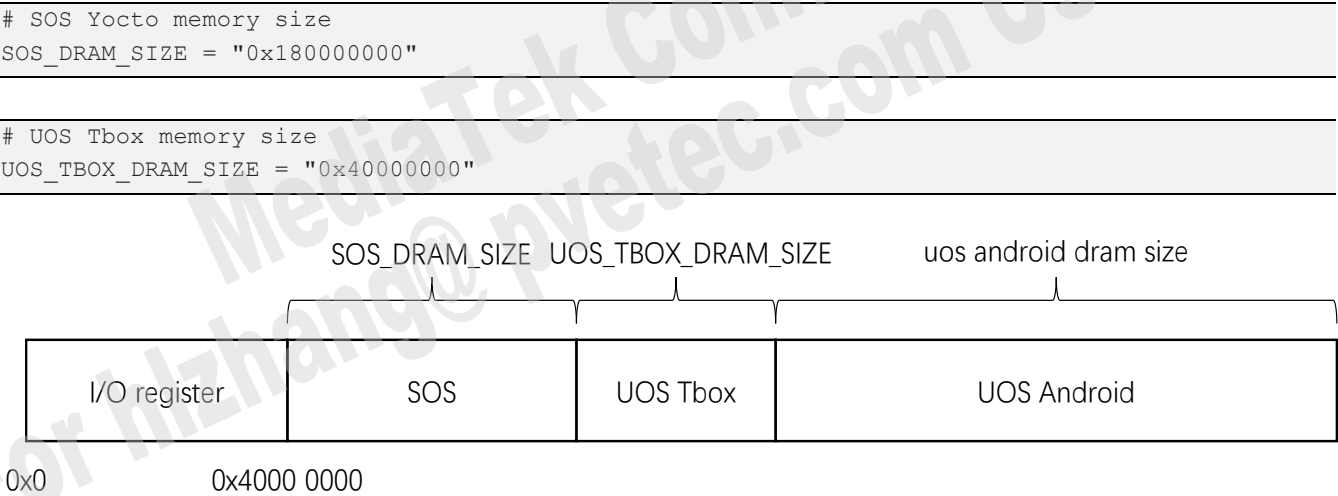


图 1-7. L+A+L 内存布局

1.4.2 调整虚拟机 CPU

虚拟机的 CPU 数量以及和物理 CPU 的绑定关系是由虚拟机的配置文件来决定的。虚拟机的配置文件路径请参考表 1-4。以下面的配置为例，总共给虚拟机分配了 8 个 vCPU。数组 `Pcpus` 决定物理 CPU 和虚拟 CPU 的绑定关系。下标表示 vCPU，值表示 pCPU。即 vCPU0 绑定了 pCPU3，vCPU1 绑定了 pCPU1，vCPU2 绑定了 pCPU2，vCPU3 绑定了 pCPU0，以此类推。请务必确保 CPU 数量等于 `Pcpus` 数组长度，如果两者不一致，会导致虚拟机无法启动。在

MT8676 平台，总共有三个 CPU 簇，pCPU0~pCPU3 属于同一簇，pCPU4~pCPU6 属于同一簇，pCPU7 属于第三个簇。请不要跨 CPU 簇绑定 CPU，这样无法发挥出 CPU 的所有性能。

```
-- cpus
uos_config:setCpus(8)
--*****

-- physical-cpus
Pcpus = {3, 1, 2, 0, 4, 5, 6, 7}
for i = 1, #Pcpus do
    uos_config:setBindcpus(Pcpus[i])
end
--*****
```

1.4.3 添加共享文件夹

如果想把 SOS 的一个文件夹共享给 UOS，可以通过 virtio 文件系统实现，以下是实现步骤。下面提到的配置文件路径，请参考表 1-4。

第一步，修改 vm_srv_cfg.pb.txt，为目标 UOS 添加 virtiofsd 后端程序。例如下方的示例代码，给 UOS T-Box 添加了一个 virtiofsd 后端程序。shared-dir 参数用于指定共享的文件夹，socket-path 和 tag 参数后面会用到。

```
vmrecords {
  name: "tbox"
  .....

  backends {
    exec: "virtiofsd"
    args: "--socket-path=/tmp/virtiofs_tbox_socket0"
    args: "--shared-dir=/mnt/vendor/nvdata"
    args: "--tag=tbox_nvdata"
    args: "--sandbox=none"
    args: "--modcaps=+sys_admin"
  }
}
```

第二步，修改 UOS 配置文件，添加如下代码，第一个参数是 tag，第二个参数是 socket 路径。这两个参数的值请参考第一步。这里以 UOS T-Box 为例，所以需要修改 uos_tbox_pv8676.lua 文件。

```
uos_config:setFs("tbox_nvdata", "/tmp/virtiofs_tbox_socket0")
```

第三步，断电重启整个系统，待 UOS 启动完成后，进入 UOS 的 shell。执行如下命令测试挂载。第三个参数 `tbbox_nvdata` 是第一步设置的 tag 参数。如果挂载成功，表示文件夹共享成功。如果失败，返回检查第一步和第二步。

```
mkdir -p /tmp/test_mount
mount -t virtiofs tbbox_nvdata /tmp/test_mount
```

第四步，设置开机挂载。对于 UOS Android，请修改 `device/mediateksample/<project>/init.project.rc` 文件。示例如下。

```
on post-fs-data
.....
#virtio-fs sample
    mkdir /data/vendor/share/
    mkdir /data/vendor/share/media/
    mount virtiofs media /data/vendor/share/media
```

对于 UOS T-Box，请修改 `meta/meta-mediatek-mt8676-hyp/recipes-core/base-files/base-files/auto8676p1_64_uos_tbbox/fstab` 文件。示例如下。

tbbox_nvdata	/mnt/vendor/nvdata	virtiofs defaults 0 0
--------------	--------------------	-----------------------

1.5 调试技巧

关于 Yocto 的调试技巧，请参考文档 *MT8676_Yocto_System_User_Manual*，关于 Android 的调试技巧，请参考文档 *MT8676_Android_System_User_Manual*。这里只介绍 Hypervisor 相关的调试技巧。

1.5.1 切换 USB

MT8676 只有一个 USB 硬件，所以只有一个操作系统能够使用 USB。对于全虚拟化架构，USB 默认在 SOS，对于部分虚拟化架构，USB 默认在 UOS Android。可以执行如下指令中的一条指令，将 USB 切换到指定系统。指令可以在任何一个系统执行。如果在 UOS Android 执行，请切换到 root 用户执行。指令执行之后，需断电重启系统，设置才会生效。

```
# 切换到 SOS
sysenv_test write USB_SWITCH Y

# 切换到 UOS Android
sysenv_test write USB_SWITCH A

# 切换到 UOS Tbox
sysenv_test write USB_SWITCH T
```

1.5.2 ADB

USB 所在的操作系统，使用物理 adb。其他两个系统使用网络 adb。表 1-10 列出了每个系统进入 adb shell 的指令。例如全虚拟化架构，USB 默认在 SOS，所以需要第一行的指令。

表 1-10. adb shell 指令

USB	SOS	UOS Android	UOS T-Box
SOS	adb -d shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS Android	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS T-Box	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d shell

1.5.3 查询虚拟机状态

在 SOS 中执行命令 `cat /sys/guest_os/android/pm_state` 查询 UOS Android 的状态，执行命令 `cat /sys/guest_os/tbox/pm_state` 查询 UOS T-Box 的状态。

1.6 开机流程

MT8676 的开机流程如图 1-8 所示。

1. 当平台上电之后，Boot ROM 会加载并跳转到 SOS LK2。SOS LK2 会完成一些基本的硬件初始化，加载 kernel、tinysys、tee、gz 等镜像，最后通过 TF-A 跳转到 nebula，开始初始化虚拟机。虚拟机在完成自身的一些初始化之后，会启动一个名为 SOS 的任务，根据相关配置文件，启动 SOS 虚拟机。
2. SOS 虚拟机按照 kernel、initramfs、systemd 的顺序依次完成初始化，关于这些模块初始化的详细说明，请参考文档 *MT8676_Yocto_System_User_Manual*。
systemd 初始化完成之后，会启动各种用户层服务，其中一个是虚拟机服务进程 nbl_vm_srv。nbl_vm_srv 会解析配置文件 vm_srv_cfg.pb.txt，根据相应的配置启动 UOS 虚拟机。关于此配置文件的说明，请参考表 1-4。
3. 对于每一个 UOS 虚拟机，nbl_vm_srv 进程会启动一个对应的虚拟机管理进程 nbl_vmm。nbl_vmm 进程会读取虚拟机配置文件 uos_alps_pv8676.lua（或 uos_tbox_pv8676.lua），根据配置创建 bootloader、初始化后端驱动，并序列化 CPU、内存、irq 等配置，传递给 nebula 启动 UOS。

4. nebula 创建好 UOS 虚拟机后，跳转到 UOS LK2，开始启动 UOS。最后，UOS 开始自己的启动流程。对于 UOS Android 的启动流程说明，请参考文档 *MT8676_Android_System_User_Manual*。对于 UOS Tbox，请参考文档 *MT8676_Yocto_System_User_Manual*。

SOS 启动完成之后，服务 `nbl_yocto_cpu_offline.service` 会按 CPU 分配策略关闭 SOS 的 vCPU。UOS Android 启动完成后，`init.project.rc` 会按 CPU 分配策略关闭 UOS Android 的 vCPU。UOS Tbox 启动完成后，会在 `initramfs` 阶段的 `uos_earlyinit` 脚本、`systemd` 阶段的 `uos_init.service` 按 CPU 分配策略关闭 UOS Tbox 的 CPU。具体分核策略请参考谦川文档：虚拟机开机-CPU 开关优化。

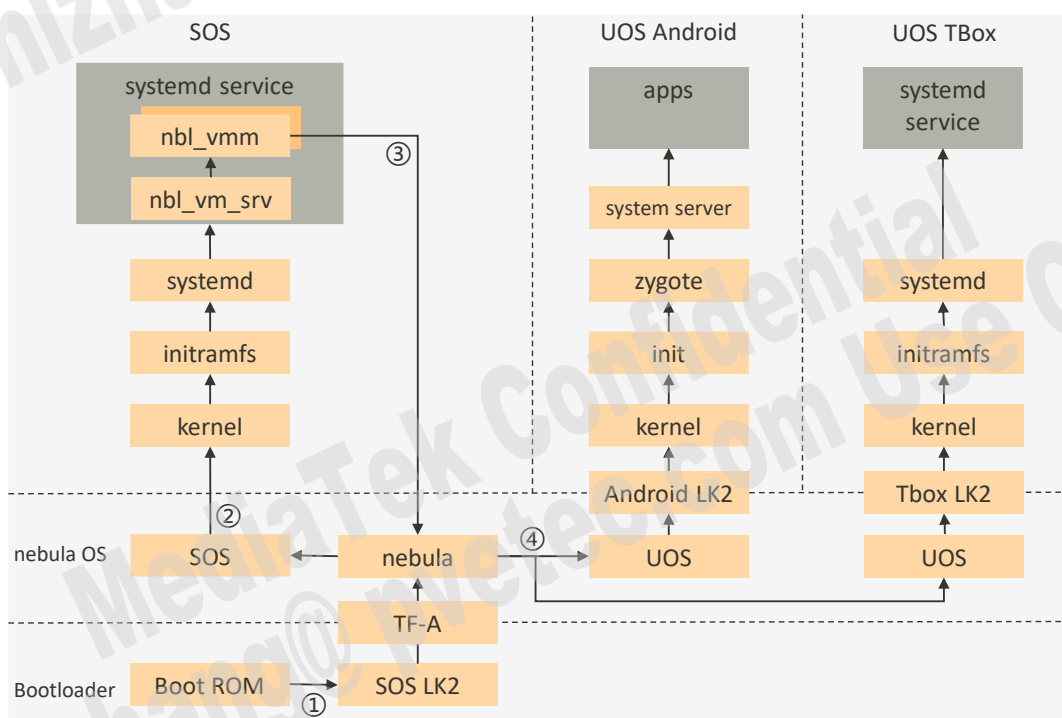


图 1-8. 开机流程

主要的开机日志如下。

```
(250613_13:18:11.624)P11 init start...
(250613_13:18:18.671)[ 0.000000][ T0] swapper: [name:setup&]Booting Linux on physical
CPU 0x0000000000 [0x411fd461]
(250613_13:18:24.312)[ 5.034945][T1700001] systemd: systemd[1]: systemd 255.4^ running in
system mode (+PAM -AUDIT -SELINUX -APPARMOR +IMA -SMACK +SECCOMP -GCRYPT -GNUTLS -OPENSSL
+ACL +BLKID -CURL -ELFUTILS -FIDO2 -IDN2 -IDN -IPTC +KMOD -LIBCRYPTSETUP +LIBFDISK -PCRE2 -
PWQUALITY -P11KIT -QRENCODE -TPM2 -BZIP2 -LZ4 -XZ -ZLIB +ZSTD -BPF_FRAMEWORK -XKBCOMMON
+UTMP +SYSVINIT default-hierarchy=unified)
(250613_13:18:35.383) Starting Nebula VM server...
(250613_13:18:35.452)[ OK ] Started Nebula VM server.
(250613_13:18:35.618)[ 16.320448] nbl_vm_srv[2341]: nbl_vm_srv I 06-11 21:36:53 2341
2341 vm_controller.cc:195] [vmid:0] StartVm enter
(250613_13:18:35.702)[ 16.344367] nbl_vm_srv[2341]: nbl_vm_srv I 06-11 21:36:53 2341
2341 vm_controller.cc:92] [vmid:0] ForkAndExec start
```

```
(250613_13:18:35.790)[ 16.433583] nbl_vm_srv[2378]: nbl_vmm E 06-11 21:36:53 2378 2378
main.cc:428] config file : /vendor/etc/hyper_android/uos_alps_pv8676.lua
(250613_13:18:36.024)[ 16.692737] nbl_vm_srv[2378]: nbl_vmm vmid:0 E 06-11 21:36:53 2378
2378 guest_config.h:125] setMemsize M: 9439M
.....
(250613_13:18:36.336)[ 17.033775] nbl_vm_srv[2378]: nbl_vmm vmid:0 I 06-11 21:36:53 2378
2378 main.cc:551] Success to attach vhost blk
(250613_13:18:36.336)[ 17.044333] nbl_vm_srv[2378]: nbl_vmm vmid:0 I 06-11 21:36:53 2378
2378 main.cc:571] Success to attach vhost user gpu device
.....
(250613_13:18:37.118)[ 17.796357] nbl_vm_srv[2378]: nbl_vmm vmid:0 E 06-11 21:36:53 2378
2378 main.cc:936] vm_ctl_svc start status 0
```

1.7 关机流程

本小节介绍了 SOS 和 UOS 的关机流程，以及 UOS 重启整个 SOC 的流程。

1.7.1 UOS 关机

UOS 关机流程如图 1-9 所示。当 UOS 用户空间触发关机或重启后，init 进程会收到相关信号，开始停止用户层服务、卸载文件系统等。这个过程取决于 init 进程的实现。

init 进程处理完成之后，调用 reboot() 系统调用进入内核空间。vmctl 内核模块用于处理虚拟机的关机。内核经过一系列的调用后，最终会调用到 vmctl 前端注册的处理函数。

对于关机，reboot() 系统调用会依次调用 __arm64_sys_reboot() -> kernel_power_off() ->

machine_power_off() -> do_kernel_power_off()，最终调用到 vmctl 的 sys_poweroff()，给 SOS 发送 VMCTL_POWER_OFF 消息。

对于重启，reboot() 系统调用会依次调用 __arm64_sys_reboot() -> kernel_restart() -> machine_restart() -> do_kernel_restart()，最终会调用到 vmctl 的 sys_restart()，给 SOS 发送 VMCTL_RESTART 消息。

vmctl 后端位于 SOS 的虚拟机管理进程 nbl_vmm。当 vmctl 后端收到消息后，会转发消息给虚拟机服务进程

nbl_vm_srv。如果收到了 VMCTL_POWER_OFF 消息，nbl_vm_srv 会直接关闭对应的 nbl_vmm 进程；如果收到了 VMCTL_RESTART 消息，会重启 nbl_vmm 进程，再次拉起 UOS。

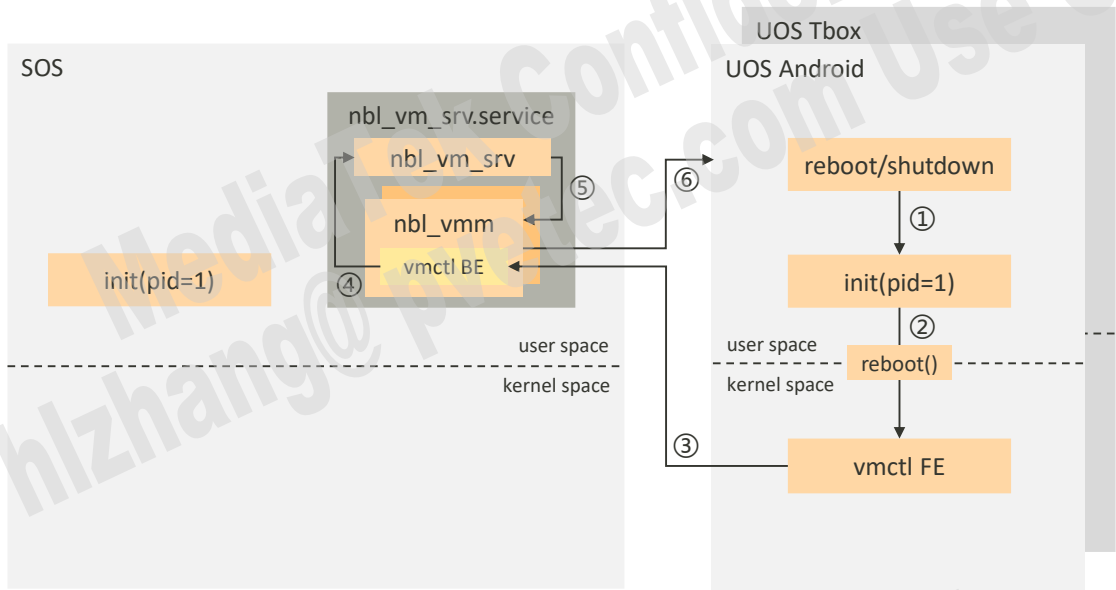


图 1-9. UOS 关机流程

以 UOS Android 重启为例，主要日志如下。

```
(250613_16:08:54.186) [ 9970.951718] nbl_vm_srv[3907]: [vmid:0]console:/ $ reboot
(250613_16:09:00.889) [ 9977.619800] nbl_vm_srv[3907]: nbl_vmm vmid:0 I 06-12 00:27:17 3907
3953 vmctl.cc:73] Host vmm recieve VMCTL_RESTART
(250613_16:09:00.889) [ 9977.620433] nbl_vm_srv[2334]: nbl_vm_srv I 06-12 00:27:17 2334
3965 vm_controller.cc:584] [vmid:0] received VmState: RESTART
(250613_16:09:00.889) [ 9977.620952] nbl_vm_srv[2334]: nbl_vm_srv I 06-12 00:27:17 2334
2363 vm_controller.cc:259] [vmid:0] send SIGTERM to pid 3907
(250613_16:09:17.702) [ 9994.438314] nbl_vm_srv[2334]: nbl_vm_srv I 06-12 00:27:34 2334
2363 vm_controller.cc:280] [vmid:0] nbl vm controller stop vm finish.
(250613_16:09:17.702) [ 9994.438558] nbl_vm_srv[2334]: nbl_vm_srv I 06-12 00:27:34 2334
2363 vm_controller.cc:286] [vmid:0] RestartVm --- StartVm
(250613_16:09:17.702) [ 9994.438683] nbl_vm_srv[2334]: nbl_vm_srv I 06-12 00:27:34 2334
2363 vm_controller.cc:195] [vmid:0] StartVm enter
(250613_16:09:17.702) [ 9994.468310] nbl_vm_srv[10533]: nbl_vmm E 06-12 00:27:34 10533 10533
main.cc:428] config file : /vendor/etc/hyper_android/uos_alps_pv8676.lua
```

1.7.2 SOS 关机

SOS 关机流程如图 1-10 所示。当 SOS 用户空间触发关机或重启后，init 进程会收到相关信号，开始停止用户层服务、卸载文件系统等。SOS 的 init 进程是 systemd，关于 systemd 的关闭流程，请参考 [System Manager Shutdown](#)^[4]。

当 nbl_vm_srv.service 收到关闭的消息后，会检查 UOS 的电源状态，如果是开机状态，则会给 UOS 发送一个关机消息，触发 UOS 的关机流程。关于 UOS 的关机流程说明，请参考 1.7.1 小节。nbl_vm_srv.service 会每隔 1s 检查一次

[4] <https://www.freedesktop.org/software/systemd/man/latest/bootup.html#System%20Manager%20Shutdown>

UOS 的电源状态，如果是开机状态，会再次发送关机消息，直到 UOS 关机完成。为了避免 UOS 关机异常阻塞 SOS 关机，nbl_vm_srv.service 最多会发送 15 次关机消息，如果超过 15 次 UOS 还没有关机完成，则 nbl_vm_srv.service 会强制关闭 UOS。

当用户空间服务关闭完成后，init 进程调用 reboot() 系统调用进入内核空间。内核空间的关闭流程与 UOS 类似，不同的是，SOS 会 smc 到 ATF，关闭或重启 soc。

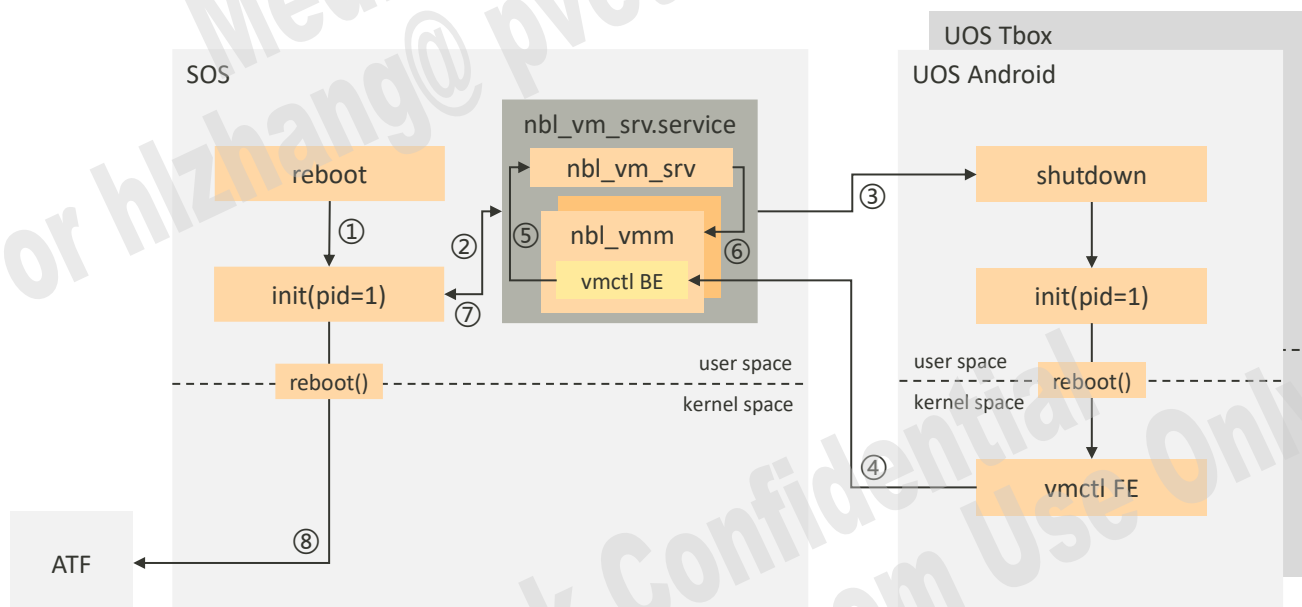


图 1-10. SOS 关机流程

SOS 关机的主要日志如下。

```
(250613_13:22:24.173)reboot
(250613_13:22:24.439)      Stopping Nebula VM server...
(250613_13:22:24.517)[ 245.148570] nbl_vm_srv[2341]: nbl_vm_srv I 06-11 21:40:41 2341
2595 server.cc:102] nbl_vm_srv dump vm info:
(250613_13:22:24.524)[ 245.160522] nbl_vm_srv[2341]: nbl_vm_srv I 06-11 21:40:41 2341
2595 server.cc:116] [{"vmid":"0","state":"Started"}, {"vmid":"1","state":"Started"}]
(250613_13:22:24.524)[ 245.185579] nbl_stop_vm[4139]: stop vm 0
(250613_13:22:24.605)[ 245.255717] nbl_stop_vm[4193]: shutdown command sent to vm with CID
3
(250613_13:22:24.605)[ 245.289480] nbl_stop_vm[4139]: Waiting for vm 0 to stop... (0/30)
(250613_13:22:24.833)[ 245.557440] nbl_stop_vm[4195]: shutdown command sent to vm with CID
4
(250613_13:22:24.913)[ 245.625417] nbl_stop_vm[4140]: Waiting for vm 1 to stop... (0/30)
.....
(250613_13:22:31.751)[ 252.448385] nbl_stop_vm[4139]: vm 0 has stopped.
(250613_13:22:33.882)[ 254.564758] nbl_stop_vm[4140]: vm 1 has stopped.
(250613_13:22:33.882)[ 254.576510] nbl_stop_vm[4138]: all vm has stopped
(250613_13:22:33.882)[ OK ] Stopped Nebula VM server.
(250613_13:22:34.831)[ OK ] Reached target System Reboot.
```

1.7.3 reboot soc

reboot soc 功能用于在 UOS 重启整个系统。此功能默认关闭。修改/vendor/etc/hyper_android/vm_srv_cfg.pb.txt 文件，将对应 UOS 的 reboot_yocto 设置为 true，可以打开此 UOS 的 reboot soc 功能。在 UOS 执行 “reboot soc” 命令，即可使用此功能。如果 reboot_yocto 是 false，则只会重启 UOS。

reboot soc 流程如图 1-11 所示。以 UOS Android 执行 reboot soc 为例，当重启的流程执行到 vmctl 前端时，vmctl 判断 reboot 指令的参数是 soc 后，会给 vmctl 后端发送 VMCTL_REBOOT_YOCTO 消息。

当虚拟机服务进程 nbl_vm_srv 收到 VMCTL_REBOOT_YOCTO，且 reboot_yocto 为 true 时，会执行 reboot 指令，触发 SOS 的重启流程，相关流程说明请参考 1.7.2 小节。有点不同的是，此时只有 UOS Tbox 处于开机状态，nbl_vm_srv.service 只会通知 UOS Tbox 关机。

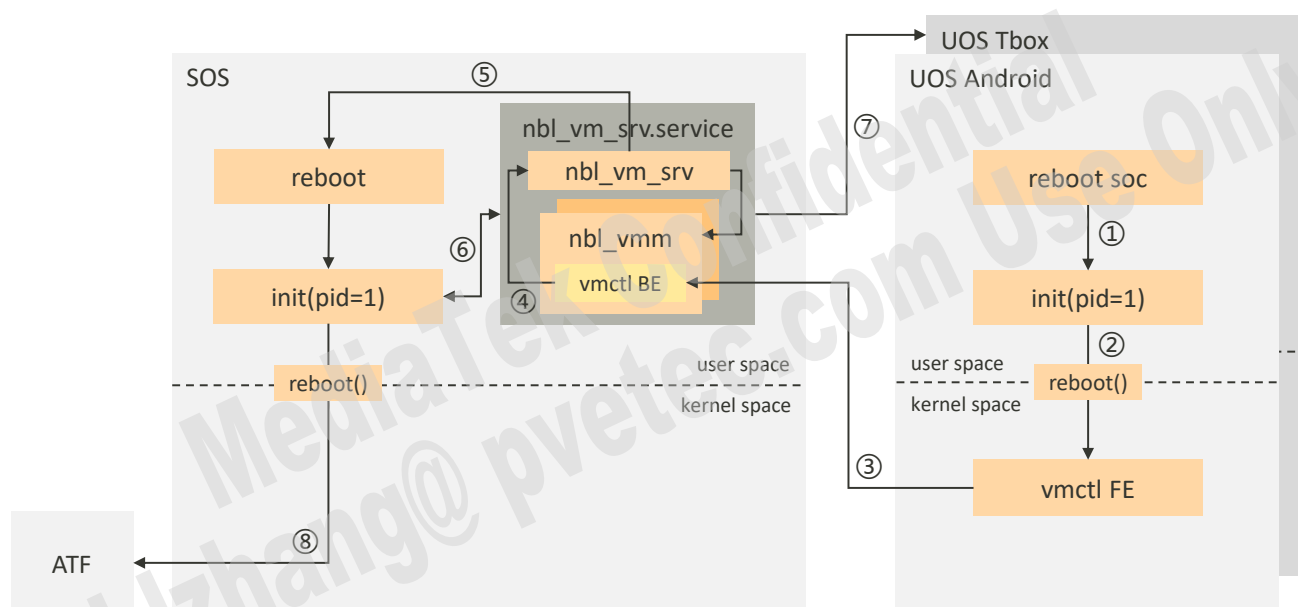


图 1-11. reboot soc 流程

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。